University of Camerino

School of Science and Technologies

Master Degree in Computer Science (Class LM-18)

Course of Knowledge Engineering

Credit card fraud detection system

Group members
Nicola Lerco (Student ID 135770)
Riccardo Peruzzi (Student ID 135786)

Supervisor
Prof. Dr. Knut Hinkelmann

A.A. 2025/2026

Contents

List of Figures

List of Tables

1. Introduction

This project concerns the development of an intelligent system for detecting fraud in credit card transactions, based on the application of knowledge representation and reasoning principles. The main objective is to implement a decision model that operates according to a risk-based approach, calculating a score for each individual financial operation and triggering appropriate responses when critical thresholds are exceeded.
The system analyzes several variables associated with each transaction and based on these inputs and predefined business rules assign partial risk scores that are aggregated into a final score, which determines whether a transaction should be approved, suspended for human review, or automatically rejected.
The project is developed incrementally across multiple tasks, each introducing new requirements and formalisms. The decision logic is first modeled using the DMN (Decision Model and Notation) standard, a specification developed by the Object Management Group (OMG) that provides a structured and interoperable way to represent repeatable business decisions. The knowledge base is then extended and reimplemented using Prolog, a logic programming language well suited for rule-based reasoning and relational queries over structured data. In the next stage, the domain knowledge is further formalized as a Knowledge Graph using Protégé: an RDF schema is defined to capture the relevant concepts and relationships, the test cases are represented as RDF instances, and SPARQL queries are used to retrieve transactions of interest. SWRL rules are additionally introduced to derive, through automated inference, whether each transaction should be classified as fraudulent.
Finally, a further extension introduces the distinction between A-transactions and B-transactions, where A-transactions are defined as those made in online stores, while B-transactions include all other transactions. This classification is implemented both in Prolog and in the Knowledge Graph, exploring the capabilities and limitations of SWRL rules and SPARQL queries in handling class complementarity under the open world assumption. The complete system thus demonstrates a progressive refinement of the fraud detection model across multiple knowledge representation paradigms, from DMN to Prolog to OWL/SWRL, showcasing the strengths and trade-offs of each formalism in the context of a real-world business rule application.

2. Requirements

The system to be developed aims to detect fraudulent credit card transactions through a risk-based approach. For each transaction, a risk score is calculated; when certain thresholds are exceeded, the transaction is either stopped for human verification or rejected outright as fraudulent.

Scenario

Every payment made with a credit card is stored as a transaction in the billing system of the bank that issued the credit card. A transaction contains and refers to a lot of data. In this project, the focus is on the following data:

- the payment amount;
- the cardholder; specifically, the name and the country where the cardholder resides;
- the merchant; specifically, the name, the country where the merchant is based, and the type of service used (online store or physical store);
- the date and time of the payment. For representing the time, a timestamp can be used, which is a number giving the time in seconds starting from 0 on 1st January 1970. For example, the timestamp of noon on 4th June 2026 is 1,780,574,400.

Rules

The rules governing this calculation are defined in the project specification and are summarised in Table 2.1.

Once the overall risk score has been calculated, the final outcome is determined as follows: if the score is less than or equal to 100, the transaction is accepted; if it is between 101 and 150, the transaction is stopped for human review; if it exceeds 150, the transaction is declined and classified as fraudulent.

Test cases

About tests, four scenarios have been defined to validate the system, as listed below. The fourth scenario is specifically designed to test the temporal rule, since it involves the same cardholder's credit card being used in two different countries within a three-minute interval.

- **Scenario 1:** Mr. Mensoni (Italy) purchases a wooden mask worth €11,234 at "World of Africa" (South Africa, physical store).

Category	Condition	Risk Points
Merchant location	Italy	+10
	Europe	+20
	Africa	+60
Cardholder location	Italy	+0
	Other European country	+10
	Africa	+20
Special case	Swiss cardholder in South Africa	+30
Amount	Less than €1,000	+10
	Up to €10,000	+60
	Above €10,000	+90
5 minutes	Less than 5 minutes between two transaction with same credit card in different physical stores	+150

Table 2.1: Risk scoring rules applied per transaction.

- **Scenario 2:** Mr. Meyer (Germany) pays €12 for a meal at "Winds of the Deserts" (Morocco, physical store).
- **Scenario 3:** Mr. Suter (Switzerland) pays €90 for a meal at "Mama Africa" (South Africa, physical store).
- **Scenario 4:** Mr. Mensoni's credit card is used again, only 3 minutes after the first purchase, at "Winds of the Deserts" (Morocco, physical store) for a meal costing 500 CHF.

2.1 Part 1: Decision Modelling with DMN

The DMN model for fraud detection is designed to put into practice the risk-based logic. In order to correctly calculate the risk score for each transaction and to determine whether a transaction should be accepted, stopped or rejected, the model must respect a number of functional and structural requirements, which are presented in detail below.

Inputs

The system requires three input variables for each transaction: the merchant's country (`Country_Merchant`), the cardholder's country (`Country_Carholder`), and the transaction amount (`Amount`).

Sub-decisions

The model must be structured into modular sub-decisions. A geographic mapping decision (`Country_Continent`) must convert raw country names into continent-level categories (specifically Italy, Europe, Africa, Switzerland, and Other) to allow continent-based risk rules to be applied uniformly. Four independent risk-scoring sub-decisions must then compute partial scores: `Merchant_Risk` based on where the card is used, `Carholder_Risk` based on where the cardholder resides, `Special_Risk` for the spe-

cific case of a Swiss cardholder transacting in South Africa, and `Amount_Risk` based on the transaction value.

Main decision

A `Total_Risk` decision must aggregate all partial scores into a single numerical risk score. A final `Result` decision must then evaluate the total score and produce one of three outcomes: approved (score ≤ 100), stopped for human review (score > 100), or rejected (score > 150).

Representability of Rule 5 in DMN

It must be discussed whether rule 5 (two physical transactions in different countries within 5 minutes) can be represented in DMN. If a solution exists, it must be created and explained; otherwise, a motivated explanation of why the rule cannot be expressed within the DMN formalism must be provided.

2.2 Part 2: Ontology

This task requires defining a graphical ontology that captures the core terminology of the domain.

Concepts and Relationships

The ontology must define the most important terms as concepts and the relationships between them as properties.

Data Exclusion

Data about transactions must not be included in the graphical ontology. Only the terminological level (T-box) must be represented.

2.3 Part 3: Prolog

A knowledge base must be implemented in Prolog that incorporates all fraud detection rules defined up to this point, including both the scoring rules from Part 1 and the temporal comparison rule (5 minutes).

Knowledge Representation

The implementation must represent transactions, cardholders, and merchants as structured facts using suitable predicates.

Test Cases

The test scenarios must be represented as Prolog facts.

Queries

Appropriate queries must be formulated to detect and classify fraudulent transactions.

2.4 Part 4: Knowledge Graph and Rules

This task requires converting the Prolog solution into a knowledge graph using Protégé and by some steps.

RDF Schema

For this step, it is necessary to develop an ontology in Protégé that defines the fundamental classes of the domain. The object properties to relate these classes and the data properties to store concrete values must also be specified.

Test Cases in RDF

Once the schema has been defined, the individuals corresponding to the scenarios provided by the text must be inserted into the ontology.

SPARQL Queries

This section should contain SPARQL queries. The first retrieves a list of all transactions in the knowledge graph. The second selects transactions whose trader is based in South Africa. The third returns transactions whose cardholders reside in a European country. Finally, the fourth extracts transactions with Italian merchants and shows the corresponding risk associated with the merchant for each.

SWRL Rules

The risk rules defined in the previous tasks should be translated into SWRL rules. Specifically, the rules for the geographic risk of the merchant and cardholder, for the amount-based risk, for the special risk involving Swiss cardholders and South African merchants, and finally the time rule (5 minutes). A sum rule combines all contributions, while three separate rules assign the final decision.

SPARQL for fraudulent transactions

After activating the Protégé reasoner to enforce SWRL rules, a SPARQL query should be written that retrieves only transactions considered fraudulent.

2.5 Part 5: A-B Transaction

At the end, the terminology has been updated to distinguish between A-transactions and B-transactions as subclasses of Transaction.
Every transaction made in an online store is classified as an A-transaction. A B-transaction is the opposite; this means that it refers to every transaction that is not an A-transaction.

Test Scenarios

The following transactions must be used for testing:

- Mr. Miller from Germany orders something for €102 from the online store "South African Master Pieces" in South Africa.
- Mr. Suter from Switzerland pays for a meal in the restaurant "Mama Africa" in South Africa for €90.

Implementation

The rules for A-transaction and B-transaction must be represented in :

- Prolog.
- It must be discussed whether it is possible to create SWRL rules to identify A-transactions and B-transactions.
- It must be discussed whether A and B transactions can be retrieved with SPARQL. If not, a motivated explanation must be provided, considering the open world assumption of OWL.

3. Decision Modelling with DMN

3.1 Decision Model

Decision Model and Notation (DMN) is a standard developed by the Object Management Group (OMG) and widely used in business analysis. It provides a structured way to represent and model repeatable decisions within organizations, ensuring that decision logic can be shared and reused across different systems and companies. By using DMN, organizations can design clear and consistent decision models that support decision-making processes and the management of business rules.

3.2 DRD overview

The DRD (decision requirements diagram) model is structured as a pipeline of sub-decisions that independently compute partial risk scores, which are then aggregated into a final risk assessment. The four input variables — `Amount`, `Country_Merchant`, `Country_Carholder`, and `TimeFlag` — feed into dedicated decision nodes, whose outputs converge into the `Total_Risk` decision. A final `Result` decision then evaluates the total score against predefined thresholds to determine the outcome of the transaction.

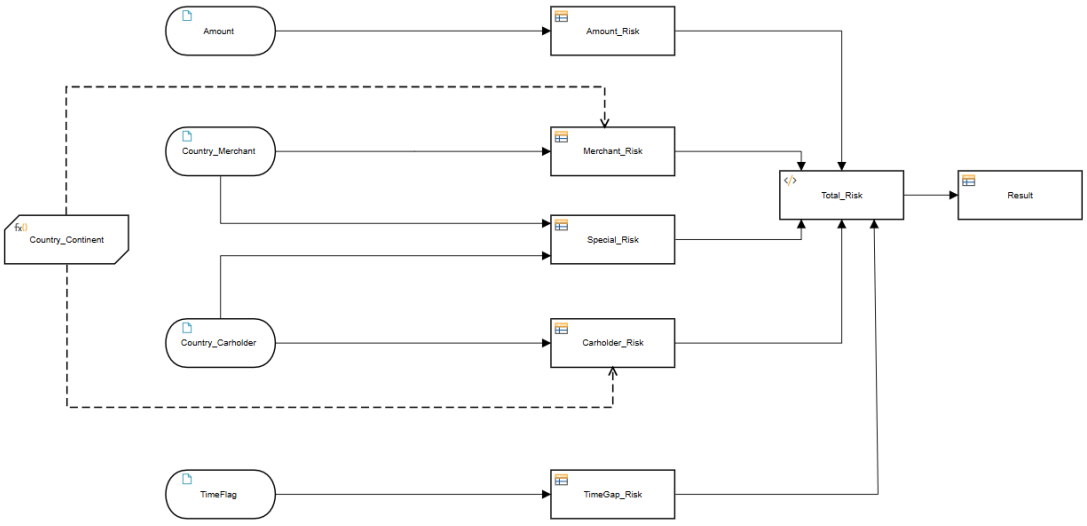

Figure 3.1: Overall structure of the DMN fraud detection model.

As shown in Figure 3.1, the model makes use of a Business Knowledge Model (BKM) called `Country_Continent`, which is invoked by the risk-scoring decisions to map raw

country names into continent-level categories.

3.3 Input variables

The model accepts four input variables per transaction. The first is `Amount`, a numerical value representing the transaction amount in euros.

The next two inputs, `Country_Merchant` and `Country_Carholder`, share a custom enumerated type called `Country`, defined to restrict the accepted values to a predefined list of valid nations. This type encompasses all European and African countries relevant to the model, as listed in Table 3.1.

The fourth input, `TimeFlag`, is a hypothetical boolean variable whose only purpose is to trigger the `TimeGap_Risk` contribution described in Section 3.5.5.

Region	**Countries**
Europe	Albania, Andorra, Austria, Belarus, Belgium, Bosnia and Herzegovina, Bulgaria, Croatia, Cyprus, Czech Republic, Denmark, Estonia, Finland, France, Germany, Greece, Hungary, Iceland, Ireland, Italy, Kosovo, Latvia, Liechtenstein, Lithuania, Luxembourg, Malta, Moldova, Monaco, Montenegro, Netherlands, North Macedonia, Norway, Poland, Portugal, Romania, San Marino, Serbia, Slovakia, Slovenia, Spain, Sweden, Switzerland, Ukraine, United Kingdom, Vatican City
Africa	Algeria, Angola, Benin, Botswana, Burkina Faso, Burundi, Cabo Verde, Cameroon, Central African Republic, Chad, Comoros, Democratic Republic of the Congo, Djibouti, Egypt, Equatorial Guinea, Eritrea, Eswatini, Ethiopia, Gabon, Gambia, Ghana, Guinea, Guinea-Bissau, Ivory Coast (Côte d'Ivoire), Kenya, Lesotho, Liberia, Libya, Madagascar, Malawi, Mali, Mauritania, Mauritius, Morocco, Mozambique, Namibia, Niger, Nigeria, Republic of the Congo, Rwanda, Sao Tome and Principe, Senegal, Seychelles, Sierra Leone, Somalia, South Africa, South Sudan, Sudan, Tanzania, Togo, Tunisia, Uganda, Zambia, Zimbabwe

Table 3.1: Countries included in the custom `Country` type.

3.4 Business Knowledge Model: Country_Continent

The `Country_Continent` Business Knowledge Model (BKM) acts as a reusable function invoked by the risk-scoring decisions. It takes a single input of type `Country` and returns a continent-level category, mapping each country to one of the two following continents: `"Europe"`, `"Africa"`.

The mapping logic is defined using hit policy **U** (Unique), meaning there will be a unique association possible applied. As shown in Figure 3.2, each country is mapped into the respective continent.

For space reasons, not all country-to-continent mappings are listed; in Figure 3.2 shows only the first 7.

U	Country_Mercant	Country_Continent	Description
	Country *"Albania", "Andorra", "Austria", "Belarus", "Belgium", "Bosnia and Herzegovina", "Bulgaria", "Croatia", "Cyprus", "Czech Republic", "Denmark", "Estonia", "Finland", "France", "Germany", "Greece", "Hungary", "Iceland", "Ireland", "Italy", "Kosovo", "Latvia", "Liechtenstein", "Lithuania", "Luxembourg", "Malta", "Moldova", "Monaco", "Montenegro", "Netherlands", "North Macedonia", "Norway", "Poland", "Portugal", "Romania", "San Marino", "Serbia", "Slovakia", "Slovenia", "Spain", "Sweden", "Switzerland", "Ukraine", "United Kingdom", "Vatican City", "Algeria", "Angola", "Benin", "Botswana", "Burkina Faso", "Burundi", "Cabo Verde", "Cameroon", "Central African Republic", "Chad", "Comoros", "Democratic Republic of the Congo", "Djibouti", "Egypt", "Equatorial Guinea", "Eritrea", "Eswatini", "Ethiopia", "Gabon", "Gambia", "Ghana", "Guinea", "Guinea-Bissau", "Ivory Coast (Côte d'Ivoire)", "Kenya", "Lesotho", "Liberia", "Libya", "Madagascar", "Malawi", "Mali", "Mauritania", "Mauritius", "Morocco", "Mozambique", "Namibia", "Niger", "Nigeria", "Republic of the Congo", "Rwanda", "Sao Tome and Principe", "Senegal", "Seychelles", "Sierra Leone", "Somalia", "South Africa", "South Sudan", "Sudan", "Tanzania", "Togo", "Tunisia", "Uganda", "Zambia", "Zimbabwe"*	*Continent&Special* *"Europe", "Africa", "Other"*	
1	"Italy"	"Europe"	
2	"Switzerland"	"Europe"	
3	"Germany"	"Europe"	
4	"Morocco"	"Africa"	
5	"South Africa"	"Africa"	
6	"Albania"	"Europe"	
7	"Andorra"	"Europe"	

Figure 3.2: Decision Table for the function that maps contry to the respective continent

3.5 Sub-decisions

This section describes the five independent sub-decisions whose partial scores are combined by the `Total_Risk` decision. The first four — `Amount_Risk`, `Merchant_Risk`, `Carholder_Risk`, and `Special_Risk` — evaluate, respectively, the transaction amount, the merchant's location, the cardholder's country of residence, and the specific Swiss-cardholder/South-African-merchant combination. The fifth, `TimeGap_Risk`, encodes the temporal proximity rule (Rule n°5) and is discussed separately at the end of this chapter, since its input depends on data external to the DMN model.

3.5.1 Amount_Risk

The `Amount_Risk` decision computes the risk contribution of the transaction amount. Figure 3.3 show how the table takes `Amount` as a numerical input and returns a partial risk score based on three mutually exclusive ranges. It uses hit policy **U** (Unique), since each possible amount falls into exactly one interval.

U	Amount	Amount_Risk	Description
	Number	*Number*	
1	<1000	10	
2	[1000..10000]	60	
3	>10000	90	

Figure 3.3: The decision table of the Amount Risk

Amounts below €1,000 contribute 10 points, amounts between €1,000 and €10,000 contribute 60 points, and amounts above €10,000 contribute 90 points.

3.5.2 Merchant_Risk

The `Merchant_Risk` decision evaluates the geographical risk associated with the merchant's location. Figure 3.4 show how it invokes the `Country_Continent` BKM on the `Country_Merchant` input to determine the continent category and returns the corresponding risk score. A **Unique (U)** hit policy is applied, ensuring that the rules are mutually exclusive and only one rule triggers for any given input. To maintain completeness, a specific rule is defined to return a risk score of 0 for cases that do not match the primary geographical criteria for eventual future implementation.

Transactions in Italy contribute 10 points, in Europe (excluding Italy) 20 points, and in Africa 60 points.

3.5.3 Carholder_Risk

The `Carholder_Risk` decision evaluates the geographical risk associated with the residence of the cardholder. It invokes the `Country_Continent` BKM on the `Country_Carholder`

U	Country_Continent(Country_Merchant)	Country_Merchant	Merchant_Risk	Description
	Continent&Special *"Europe", "Africa", "Other"*	*Country* *"Albania", "Andorra", "Austria", "Belarus", "Belgium", "Bosnia and Herzegovina", "Bulgaria", "Croatia", "Cyprus", "Czech Republic", "Denmark", "Estonia", "Finland", "France", "Germany", "Greece", "Hungary", "Iceland", "Ireland", "Italy", "Kosovo", "Latvia", "Liechtenstein", "Lithuania", "Luxembourg", "Malta", "Moldova", "Monaco", "Montenegro", "Netherlands", "North Macedonia", "Norway", "Poland", "Portugal", "Romania", "San Marino", "Serbia", "Slovakia", "Slovenia", "Spain", "Sweden", "Switzerland", "Ukraine", "United Kingdom", "Vatican City", "Algeria", "Angola", "Benin", "Botswana", "Burkina Faso", "Burundi", "Cabo Verde", "Cameroon", "Central African Republic", "Chad", "Comoros", "Democratic Republic of the Congo", "Djibouti", "Egypt", "Equatorial Guinea", "Eritrea", "Eswatini", "Ethiopia", "Gabon", "Gambia", "Ghana", "Guinea", "Guinea-Bissau", "Ivory Coast (Côte d'Ivoire)", "Kenya", "Lesotho", "Liberia", "Libya", "Madagascar", "Malawi", "Mali", "Mauritania", "Mauritius", "Morocco", "Mozambique", "Namibia", "Niger", "Nigeria", "Republic of the Congo", "Rwanda", "Sao Tome and Principe", "Senegal", "Seychelles", "Sierra Leone", "Somalia", "South Africa", "South Sudan", "Sudan", "Tanzania", "Togo", "Tunisia", "Uganda", "Zambia", "Zimbabwe"*	*Number*	
1	"Europe"	"Italy"	10	
2	"Europe"	not("Italy")	20	
3	"Africa"	-	60	
4	"Other"	-	0	

Figure 3.4: The decision table for manage the "Merchant Risk"

input and returns a partial risk score. Hit policy **U** (Unique) is applied.

	inputs		outputs	annotations
U	Country_Continent(Country_Carholder)	Country_Carholder	Carholder_Risk	Description
	Continent&Special *"Europe", "Africa", "Other"*	*Country* *"Albania", "Andorra", "Austria", "Belarus", "Belgium", "Bosnia and Herzegovina", "Bulgaria", "Croatia", "Cyprus", "Czech Republic", "Denmark", "Estonia", "Finland", "France", "Germany", "Greece", "Hungary", "Iceland", "Ireland", "Italy", "Kosovo", "Latvia", "Liechtenstein", "Lithuania", "Luxembourg", "Malta", "Moldova", "Monaco", "Montenegro", "Netherlands", "North Macedonia", "Norway", "Poland", "Portugal", "Romania", "San Marino", "Serbia", "Slovakia", "Slovenia", "Spain", "Sweden", "Switzerland", "Ukraine", "United Kingdom", "Vatican City", "Algeria", "Angola", "Benin", "Botswana", "Burkina Faso", "Burundi", "Cabo Verde", "Cameroon", "Central African Republic", "Chad", "Comoros", "Democratic Republic of the Congo", "Djibouti", "Egypt", "Equatorial Guinea", "Eritrea", "Eswatini", "Ethiopia", "Gabon", "Gambia", "Ghana", "Guinea", "Guinea-Bissau", "Ivory Coast (Côte d'Ivoire)", "Kenya", "Lesotho", "Liberia", "Libya", "Madagascar", "Malawi", "Mali", "Mauritania", "Mauritius", "Morocco", "Mozambique", "Namibia", "Niger", "Nigeria", "Republic of the Congo", "Rwanda", "Sao Tome and Principe", "Senegal", "Seychelles", "Sierra Leone", "Somalia", "South Africa", "South Sudan", "Sudan", "Tanzania", "Togo", "Tunisia", "Uganda", "Zambia", "Zimbabwe"*	*Number*	
1	"Europe"	"Italy"	0	
2	"Europe"	not("Italy")	10	
3	"Africa"	-	20	
4	"Other"	-	0	

Figure 3.5: The decision table of Carholder Risk

Italian cardholders contribute 0 points, European cardholders 10 points, and African cardholders 20 points. Any other cardholder location also returns 0.

3.5.4 Special_Risk

The `Special_Risk` decision in Figure 3.6 captures a specific business rule that cannot be expressed through continent-level aggregation alone: a Swiss cardholder using their card in South Africa immediately adds 30 points to the risk score. It takes both `Country_Merchant` and `Country_Carholder` as direct inputs, without invoking the BKM. Hit policy **F** (First) is used, with a catch-all rule returning 0 for all other combinations.

	Country_Merchant	Country_Carholder	Special_Risk	Description
F	*Country* *"Albania", "Andorra", "Austria", "Belarus", "Belgium", "Bosnia and Herzegovina", "Bulgaria", "Croatia", "Cyprus", "Czech Republic", "Denmark", "Estonia", "Finland", "France", "Germany", "Greece", "Hungary", "Iceland", "Ireland", "Italy", "Kosovo", "Latvia", "Liechtenstein", "Lithuania", "Luxembourg", "Malta", "Moldova", "Monaco", "Montenegro", "Netherlands", "North Macedonia", "Norway", "Poland", "Portugal", "Romania", "San Marino", "Serbia", "Slovakia", "Slovenia", "Spain", "Sweden", "Switzerland", "Ukraine", "United Kingdom", "Vatican City", "Algeria", "Angola", "Benin", "Botswana", "Burkina Faso", "Burundi", "Cabo Verde", "Cameroon", "Central African Republic", "Chad", "Comoros", "Democratic Republic of the Congo", "Djibouti", "Egypt", "Equatorial Guinea", "Eritrea", "Eswatini", "Ethiopia", "Gabon", "Gambia", "Ghana", "Guinea", "Guinea-Bissau", "Ivory Coast (Côte d'Ivoire)", "Kenya", "Lesotho", "Liberia", "Libya", "Madagascar", "Malawi", "Mali", "Mauritania", "Mauritius", "Morocco", "Mozambique", "Namibia", "Niger", "Nigeria", "Republic of the Congo", "Rwanda", "Sao Tome and Principe", "Senegal", "Seychelles", "Sierra Leone", "Somalia", "South Africa", "South Sudan", "Sudan", "Tanzania", "Togo", "Tunisia", "Uganda", "Zambia", "Zimbabwe"*	*Country* *"Albania", "Andorra", "Austria", "Belarus", "Belgium", "Bosnia and Herzegovina", "Bulgaria", "Croatia", "Cyprus", "Czech Republic", "Denmark", "Estonia", "Finland", "France", "Germany", "Greece", "Hungary", "Iceland", "Ireland", "Italy", "Kosovo", "Latvia", "Liechtenstein", "Lithuania", "Luxembourg", "Malta", "Moldova", "Monaco", "Montenegro", "Netherlands", "North Macedonia", "Norway", "Poland", "Portugal", "Romania", "San Marino", "Serbia", "Slovakia", "Slovenia", "Spain", "Sweden", "Switzerland", "Ukraine", "United Kingdom", "Vatican City", "Algeria", "Angola", "Benin", "Botswana", "Burkina Faso", "Burundi", "Cabo Verde", "Cameroon", "Central African Republic", "Chad", "Comoros", "Democratic Republic of the Congo", "Djibouti", "Egypt", "Equatorial Guinea", "Eritrea", "Eswatini", "Ethiopia", "Gabon", "Gambia", "Ghana", "Guinea", "Guinea-Bissau", "Ivory Coast (Côte d'Ivoire)", "Kenya", "Lesotho", "Liberia", "Libya", "Madagascar", "Malawi", "Mali", "Mauritania", "Mauritius", "Morocco", "Mozambique", "Namibia", "Niger", "Nigeria", "Republic of the Congo", "Rwanda", "Sao Tome and Principe", "Senegal", "Seychelles", "Sierra Leone", "Somalia", "South Africa", "South Sudan", "Sudan", "Tanzania", "Togo", "Tunisia", "Uganda", "Zambia", "Zimbabwe"*	*Number*	
1	"South Africa"	"Switzerland"	30	
2	-	-	0	

Figure 3.6: The decision table for managing special risks

3.5.5 TimeGap_Risk

The `TimeGap_Risk` decision, shown in Figure 6.9, translates the boolean `TimeFlag` input into a risk contribution. If `TimeFlag` is `true`, 150 points are added to the score, immediately pushing the transaction above the rejection threshold regardless of the other contributions; if `false`, the contribution is 0. Hit policy **U** (Unique) is applied.

Unlike the other four sub-decisions, `TimeGap_Risk` does not operate on data already available within the DMN model: the value of `TimeFlag` must be computed externally. The rationale behind this design choice, together with the proposed architecture, is discussed in Section 3.8.

3.5.6 Main decision: Total_Risk

The `Total_Risk` decision at Figure 3.8 does not use a decision table. Instead, it is implemented as a literal expression that aggregates the five partial risk scores produced by the sub-decisions described above into a single numerical value:

```
Amount_Risk + Merchant_Risk + Carholder_Risk + Special_Risk +
                        TimeGap_Risk
```

This design keeps the aggregation logic minimal and transparent, delegating all rule-based reasoning to the upstream sub-decisions.

U	TimeFlag	TimeGap_Risk	Description
	Boolean	*Number*	
1	true	150	
2	false	0	

Figure 3.7: The decision table for assigning the designed score to time risks

Total_Risk
Number

```
Amount_Risk+Merchant_Risk+Carholder_Risk+Special_Risk+TimeGap_Risk
```

Figure 3.8: The total risk literal for parameters summation

3.6 Final decision: Result

The `Result` decision in Figure 3.9 evaluates the aggregated `Total_Risk` score and produces a final outcome for the transaction. It uses hit policy **U** (Unique), since the three score ranges are mutually exclusive and exhaustive.

U	Total_Risk	Result	Description
	Number	*Any*	
1	<=100	"Accepted"	
2	(100..150]	"Blocked"	
3	>150	"Rejected"	

Figure 3.9: The decision table who decide whether a transaction is fraudolent or not

A transaction is accepted when the total risk score does not exceed 100 points. Scores between 101 and 150 points trigger a block, suspending the transaction for human review. Scores above 150 points result in an automatic rejection.

3.7 Test scenarios and validation

The model was validated against three predefined scenarios. For each case, the partial risk scores produced by the sub-decisions are summed into a final `Total_Risk` value, which is then evaluated by the `Result` decision. In all three scenarios, `TimeFlag` is assumed to be `false`, since each one considers a single, isolated transaction; the temporal rule is therefore not triggered and `TimeGap_Risk` contributes 0 in every case.

Scenario 1: Mr. Mensoni

Mr. Mensoni is an Italian cardholder purchasing a wooden mask at a physical store in South Africa for €11,234.

Sub-decision	**Score**
`Merchant_Risk` (South Africa → Africa)	60
`Carholder_Risk` (Italy)	0
`Special_Risk` (Italy + South Africa)	0
`Amount_Risk` (€11,234 > €10,000)	90
`TimeGap_Risk` (TimeFlag = false)	0
Total_Risk	**150**

Table 3.2: Risk score breakdown for Mr. Mensoni.

The total score of 150 falls within the range (100..150], resulting in a `Blocked` outcome.

Scenario 2: Mr. Meyer

Mr. Meyer is a German cardholder paying for a meal in Morocco for €12.

Sub-decision	**Score**
`Merchant_Risk` (Morocco → Africa)	60
`Carholder_Risk` (Germany → Europe)	10
`Special_Risk` (Germany + Morocco)	0
`Amount_Risk` (€12 < €1,000)	10
`TimeGap_Risk` (TimeFlag = false)	0
Total_Risk	**80**

Table 3.3: Risk score breakdown for Mr. Meyer.

The total score of 80 does not exceed 100, resulting in an `Accepted` outcome.

Scenario 3: Mr. Suter

Mr. Suter is a Swiss cardholder paying for a meal in South Africa for €90.

The total score of 100 satisfies the condition $\leq$ 100, resulting in an `Accepted` outcome. Note that the `Special_Risk` rule contributes 30 additional points due to the specific combination of a Swiss cardholder transacting in South Africa, which would otherwise have gone undetected by continent-level rules alone.

Sub-decision	**Score**
`Merchant_Risk` (South Africa → Africa)	60
`Carholder_Risk` (Switzerland → Other)	0
`Special_Risk` (Switzerland + South Africa)	30
`Amount_Risk` (€90 < €1,000)	10
`TimeGap_Risk` (TimeFlag = false)	0
Total_Risk	**100**

Table 3.4: Risk score breakdown for Mr. Suter.

3.8 Rule n°5 implementation

As anticipated in Section 3.5.5, the "5-minute rule" cannot be expressed through a standard DMN decision table alone, because DMN evaluates each transaction independently from any other event. The rule, however, requires comparing two distinct transactions performed by the same cardholder over time, verifying whether they occur in different countries within an interval of less than five minutes. This kind of historical, cross-record temporal reasoning falls outside the scope of standard DMN decision tables and therefore requires an external mechanism, such as a rule engine or a logic-based system.

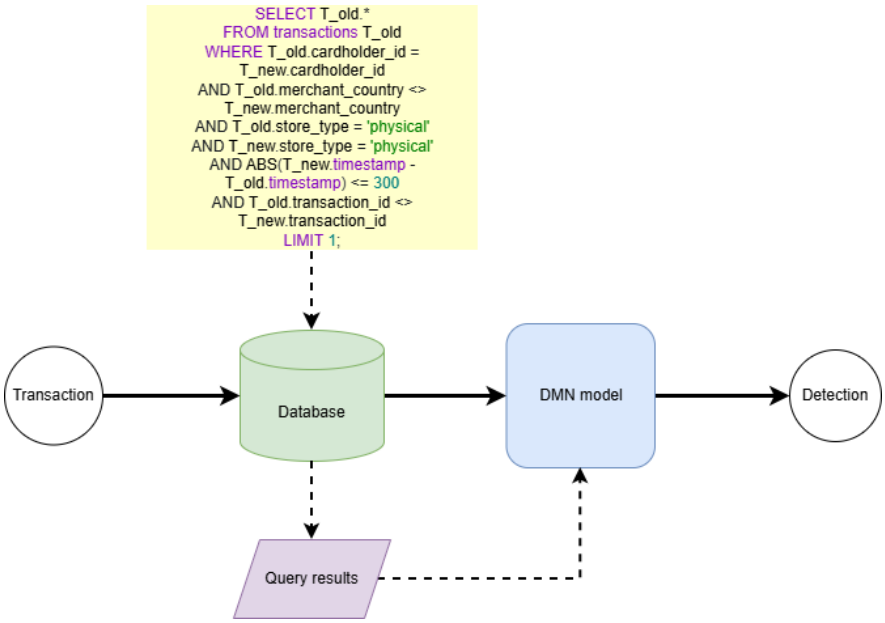

Figure 3.10: Possible solution architecture

The solution adopted to overcome this limitation integrates the DMN model with an external database through a query-based pipeline. When a new transaction is received, before being passed to the DMN engine, a query is executed on the historical transaction database (as shown in Figure 3.10). This query checks whether the same cardholder has made another transaction at a physical store in a different country within a time window of 300 seconds, by comparing the timestamps of the two transactions. The presence or absence of such a geographical conflict is then passed to the DMN model as the `TimeFlag` boolean input introduced in Section 3.5.5. In this way, the DMN model retains its role as a pure decision engine, while the database handles the historical

context, making the overall solution both scalable and maintainable.

If the database query returns at least one matching record, `TimeFlag` is set to `true` and passed as input to the DMN engine. As shown in the `TimeGap_Risk` decision table (Figure 6.9), this immediately adds 150 points to the risk score, regardless of the other input values, causing the transaction to be declined.

4. Ontology

4.1 Ontology

The figure 4.1 represents a semantic model designed to support fraud detection in financial transactions by organizing the main entities, attributes, and relationships involved in a transaction monitoring system. The central element of the model is the Transaction entity, which connects a specific Cardholder, Merchant, and Credit Card, allowing financial events to be analyzed together with their contextual information. Each transaction includes attributes such as TransactionID, Timestamp, Amount, RiskScore, and Decision, which describe both operational transaction data and the results of fraud evaluation processes.

The Cardholder entity represents the customer performing a transaction and contains descriptive information such as identifiers, names, and geographic location. The introduction of the Credit_card entity extends the model by explicitly representing payment card information through attributes such as CardNumber and by linking the card to its owner through the hasCardholder relationship. This addition improves the representation of transaction dynamics.
The Merchant entity models the commercial actor receiving the payment and includes information related to merchant identity, category, and geographic location. Geographic knowledge is represented through the classes Country and Continent, connected through hierarchical relationships that capture regional and cross-border transaction patterns. Countries such as Italy, Germany, Switzerland, Morocco, South Africa, and Tanzania are associated with continents including Europe and Africa, enabling contextual reasoning based on geographical risk factors.
The model combines relational and semantic representations by integrating entities, attributes, and graph-based connections such as hasMerchant, hasCreditCard, hasCardholder, locatedIn, and belongsToContinent. This structure helps formalize the relationships used in fraud detection rules while also providing a clear visualization of behavioral and contextual dependencies between transactions, users, merchants, and geographic regions.
Overall, the model provides an extensible and structured representation of the financial transaction domain, supporting both the visualization of the system architecture and the formalization of reasoning processes used for fraud detection and risk assessment.

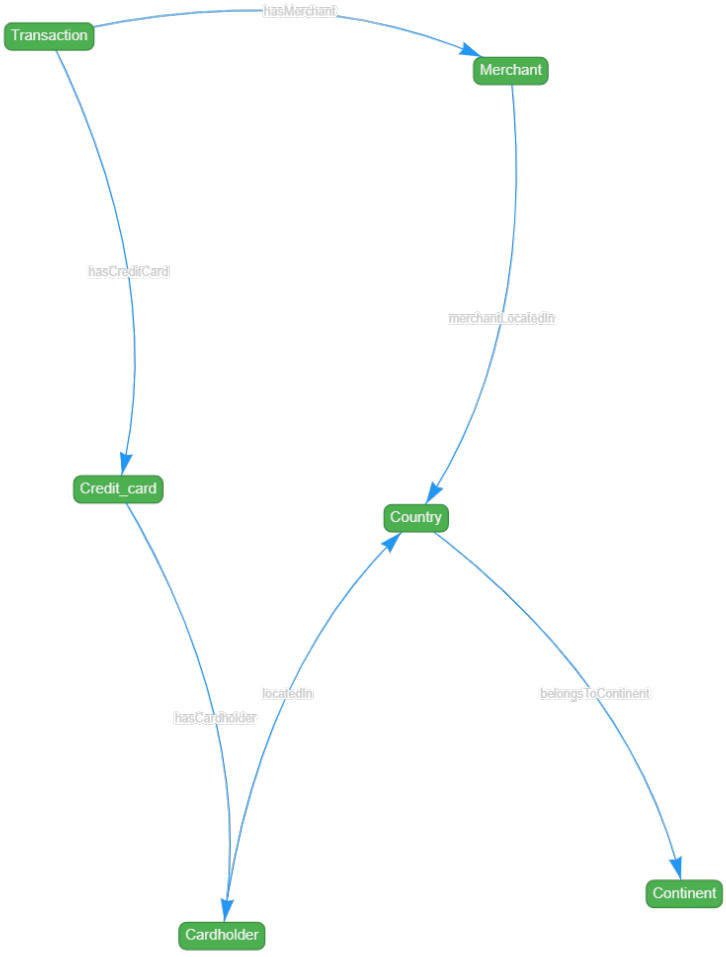

Figure 4.1: Ontology

5. Prolog

5.1 Prolog knowledge base implementation

5.1.1 General organization

The Prolog implementation is divided into two main parts: a set of facts, which declare all ground assertions about countries, cardholders, merchants, and transactions; and a set of rules, which encode the reasoning logic used to compute risk scores and classify transactions.

5.1.2 Assertions / Facts

Country predicates

The association between a country and its continent is represented through the predicate `belongs/2`, which takes a country and its corresponding continent as arguments.

```
belongs(albania, europe).
belongs(andorra, europe).
...
belongs(uganda, africa).
belongs(zambia, africa).
belongs(zimbabwe, africa).
```

Transactions and stakeholders

To simulate a database of cardholders, merchants, and transactions, three predicates are defined: `cardholder/3`, `merchant/4`, and `transaction/5`.

```
cardholder(c1, mensoni, italy).
cardholder(c2, meyer, germany).
cardholder(c3, suter, switzerland).
cardholder(c4, abba, tanzania).
```

The first argument serves as a unique identifier, used to reference the cardholder in other predicates. The remaining arguments represent the cardholder's name and country of residence.

Each cardholder is associated with a credit card number through the `creditcard/2` predicate:

```
creditcard(c1, 1111).
```

```
creditcard(c2, 2222).
creditcard(c3, 3333).
creditcard(c4, 4444).
```

The first argument is the cardholder identifier and the second is the credit card number. This indirection layer decouples the transaction records from the cardholder identity: transactions reference the card number rather than the cardholder directly, reflecting a more realistic model in which a payment instrument is the observable artifact at transaction time.

```
merchant(m1, worldofafrica, south_africa, physical).
merchant(m2, windsofthedeserts, morocco, physical).
merchant(m3, mamaafrica, south_africa, physical).
```

The merchant predicate follows the same structure, with the first argument acting as identifier. The fourth argument specifies the type of store (physical or online) which is required by the temporal comparison rule introduced in Rule 5. Each of the following transactions represents an assignment's case.

First transaction Mr. Mensoni from Italy, who purchases an expensive wooden mask in a physical store called "World of Africa" in South Africa for €11,234.

```
transaction(tx1, 1111, m1, 11234, 1715097900).
```

Second transaction Mr. Meyer from Germany, who pays for a meal worth €12 at the restaurant "Winds of the Deserts" in Morocco.

```
transaction(tx2, 2222, m2, 12, 1715097950).
```

Third transaction Mr. Suter from Switzerland pays for a meal at the restaurant "Mama Africa" in South Africa for €90.

```
transaction(tx3, 3333, m3, 90, 1715098950).
```

Fourth transaction Mr. Mensoni's credit card is used 5 minutes after the purchase of the wooden mask at the restaurant "Winds of the Deserts" in Morocco for a meal costing €12.

```
transaction(tx4, 1111, m2, 12, 1715098200).
```

Fifth transaction A transaction in which everything proceeds correctly. The scenario is the same as transaction four, but with a legitimate time window (> 5 min): Mr. Mensoni's credit card is used more than 5 minutes after the purchase of the wooden mask at the restaurant "Winds of the Deserts" in Morocco for a meal costing €12.

```
transaction(tx5, 1111, m2, 12, 1715098950).
```

Each transaction is identified by a unique ID and references a credit card number and a merchant by their respective identifiers. The cardholder identity is not stored directly in the transaction; it is instead resolved at inference time by joining the credit card number with the `creditcard/2` predicate. Transaction `tx5` is not part of the original assignment scenarios, but was added to test the temporal comparison rule: it occurs 1050 seconds later and therefore falls outside the 5-minute window.

5.1.3 Base rules

This section describes the inference rules defined to solve the task.

An early implementation included a `continent/2` predicate that wrapped `belongs/2` to map countries to continents, mirroring the role of the `Country_Continent` BKM in the DMN model. However, this predicate was later removed, as the risk rules can query `belongs/2` directly without an intermediate mapping step.

The risk-scoring predicates follow a uniform pattern: a set of clauses match specific conditions and return the corresponding score, while a final catch-all clause using the anonymous variable _ returns 0 for any case not explicitly covered.

```
merchant_risk(Country, 10) :-
    Country = italy.
merchant_risk(Country, 20) :-
    Country \= italy,
    belongs(Country, europe), !.
merchant_risk(Country, 60) :-
    belongs(Country, africa), !.
merchant_risk(_,0).


cardholder_risk(Country, 0):-
    Country = italy.
cardholder_risk(Country, 10):-
    Country \= italy,
    belongs(Country, europe), !.
cardholder_risk(Country, 20) :-
    belongs(Country, africa), !.
cardholder_risk(_,0).


amount_risk(Amount, 10):-
    Amount < 1000,!.
amount_risk(Amount, 60):-
    Amount >= 1000,
    Amount =< 10000,!.
amount_risk(Amount, 90):-
    Amount > 10000,!.


special_risk(Cardholder, Merchant, 30) :-
    Cardholder = switzerland,
    Merchant = south_africa,!.
special_risk(_,_,0).
```

The cut operator `!` is applied in every clause where backtracking could produce multiple results. Once a matching clause is found, the cut prevents Prolog from exploring further alternatives, ensuring that each predicate returns exactly one risk score per invocation. Note that `merchant_risk` and `cardholder_risk` do not require a cut in the first clause, since the explicit equality check `Country = italy` already makes that branch deterministic; the cuts in the subsequent clauses are sufficient to prevent unwanted backtracking into the catch-all.

Transaction comparison

Regarding the constraint proposed in the assignment, the `score_risk_comparison/2` rule is designed to identify suspicious behavioural patterns related to the temporal proximity of transactions. In particular, the rule verifies whether two different transactions are associated with the same credit card number, are performed in physical stores located in different countries, and occur within a time interval of at most 300 seconds (5 minutes). This condition represents a potentially fraudulent situation, since it would be highly unlikely for the same cardholder to physically perform transactions in different countries within such a short amount of time.

```
score_risk_comparison(T1, T2):-
    transaction(T1, Cardnumber, Merchant1, _, Time1),
    transaction(T2, Cardnumber, Merchant2, _, Time2),
    merchant(Merchant1, _, Country1, physical),
    merchant(Merchant2, _, Country2, physical),
    T1 \= T2,
    Country1 \= Country2,
    D is abs(Time1-Time2),
    D =< 300.

time_comparison(T, 150) :-
    transaction(T,_,_,_,_),
    score_risk_comparison(T,_),!.

time_comparison(_,0).
```

The `time_comparison/2` rule builds upon this predicate to incorporate the detected anomaly into the overall fraud evaluation process. When a transaction satisfies the suspicious pattern identified by `score_risk_comparison/2`, the rule assigns an additional risk contribution of 150 points to that transaction. The shared `Cardnumber` variable is used to correlate the two transactions: since transaction records reference the credit card number rather than the cardholder identifier directly (see Section 5.1.2), the comparison operates at the level of the payment instrument, which is the observable link between any two events at transaction time. The anonymous variable in `score_risk_comparison(T,_)` means that the rule succeeds if *any* second transaction exists that forms a suspicious pair with `T`. The cut operator (`!`) ensures that, once such a pattern has been detected, Prolog stops searching for further matching solutions and returns a single risk contribution. If no suspicious temporal pattern is found, the rule assigns a value of 0, meaning that no additional temporal risk is added to the final score.

Summation and detection predicates

Finally, the `score_risk/2` rule is responsible for computing the overall risk score associated with a transaction. It retrieves all relevant information about the transaction — the credit card number, the merchant, the amount, and the countries involved — and aggregates the partial risk contributions produced by the previously defined rules: cardholder risk, merchant risk, amount risk, special geographical risk, and the temporal comparison penalty. The cardholder is identified in two steps: first, the credit card number stored in the transaction is matched against `creditcard/2` to recover the cardholder identifier; then, `cardholder/3` is used to retrieve the country of residence. This two-step resolution keeps transaction records free of direct cardholder references, consistent with the data model described above. The final score therefore represents a comprehensive evaluation obtained by combining all these independent factors.

```
score_risk(TransactionID, Score) :-
    transaction(TransactionID, Cardnumber, MerchantID, Amount, _),
    creditcard(CardholderID, Cardnumber),
    cardholder(CardholderID, _, CardholderCountry),
    merchant(MerchantID, _, MerchantCountry, _),
    cardholder_risk(CardholderCountry, CR),
    merchant_risk(MerchantCountry, MR),
    amount_risk(Amount, AR),
    special_risk(CardholderCountry, MerchantCountry, SR),
    time_comparison(TransactionID, TC),
    Score is CR + MR + AR + SR + TC.
```

Once the total risk score has been calculated, the `detect/3` rule classifies the transaction into one of the three possible outcomes of the fraud detection system. A transaction is considered *accepted* when the score is less than or equal to 100, *stopped* when the score is strictly between 101 and 149, and *rejected* when the score is greater than or equal to 150. Note that `score_risk/2` is called independently in each clause; this is intentional, as each clause represents a mutually exclusive decision branch and the repeated call ensures that the score is bound within the correct range before the decision is made. This final rule represents the decision-making component of the model, translating the numerical risk evaluation into a concrete operational decision.

```
detect(TransactionID, accepted, Score):-
    score_risk(TransactionID, Score),
    Score =< 100.

detect(TransactionID, stopped, Score):-
    score_risk(TransactionID, Score),
    Score > 100,
    Score < 150.

detect(TransactionID, rejected, Score):-
    score_risk(TransactionID, Score),
    Score >= 150.
```

Overall, this structure allows the knowledge base to clearly separate static domain information from the reasoning mechanisms used for fraud detection. By combining

declarative facts with logical rules, the system is able to model the transaction domain in a modular and extensible way, while automatically deriving risk evaluations and final decisions through inference. This organisation also improves readability and maintainability, making it easier to extend the model with additional rules or new fraud detection patterns in the future.

5.2 Queries

The only requirement for querying the system is to use the `detect/3` predicate in the following form:

```
detect(TransactionID, Result, Score)
```

To check whether a specific transaction is legitimate, the transaction identifier must be provided as a constant in the first argument, with the second and third arguments as free variables, as in the following example:

```
detect(tx1, X, Y).
```

This query will bind the variable X to the classification result and Y to the corresponding risk score.

Query	**Result**	**Score**
detect(tx1,X,Y)	rejected	300
detect(tx2,X,Y)	accepted	80
detect(tx3,X,Y)	stopped	110
detect(tx4,X,Y)	rejected	220
detect(tx5,X,Y)	accepted	70

Table 5.1: Detection results and corresponding scores.

To retrieve all transactions in a given state, the same `detect/3` predicate can be used with the transaction identifier as a free variable:

```
detect(X, accepted, Y).
detect(X, stopped, Y).
detect(X, rejected, Y).
```

This will return all transactions in the requested state together with their scores.

6. Knowledge Graph and Rules

6.1 Define RDF schema

A Protégé ontology (Figure 6.1) has been developed in fraud detection systems that defines an RDF schema capable of representing all the elements involved in the fraud detection process. The main classes that make up the scheme are six: `Transaction` for transactions, `Cardholder` for cardholders, `Creditcard` for credit cards, `Merchant` for merchants, `Country` for countries, and `Continent` for continents.

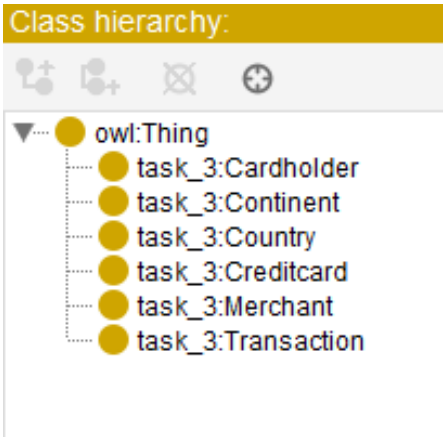

Figure 6.1: The class hierarchy created in the framework protegee

These classes are connected to each other via object properties that express the meaningful relationships of the domain. Specifically, a transaction is associated with a card via `hasCreditcard` and an merchant via `hasMerchant`; the card is in turn linked to its holder via `hasCardholder`; the cardholder resides in a country via `locatedIn`, while the merchant is located in a country via `merchantLocatedIn`; finally, each country belongs to a continent via `belongsToContinent`. Figure 6.2 shows the in-app hierarchy.

To store concrete data for transactions and other items, several data properties have been defined: `Amount` and `Timestamp` record the amount and time of each transaction, respectively; `StoreType` indicates whether the merchant operates in a physical store or online; and various properties such as `CardholderName`, `MerchantName`, `CountryName`, and `ContinentName` retain descriptive names. As shown in Figure 6.3 the schema also includes properties designed to accommodate the intermediate and final results of the risk calculation, namely`MerchantRisk`, `CardholderRisk`, `AmountRisk`, `SpecialRisk`, `TimeRisk`, `RiskScore`, and `decision`. All properties are declared with their respective domains and ranges, ensuring that each value

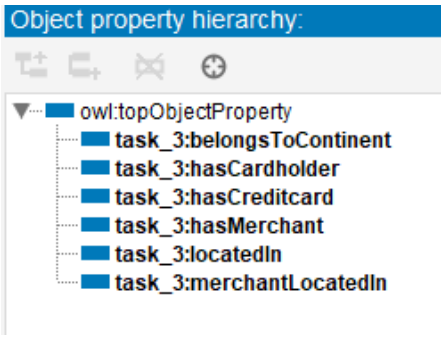

Figure 6.2: The property hierarchy created in the framework protegee

is placed in the correct class. This scheme therefore provides a formal, complete and reusable basis for representing all the information needed by the fraud detection system.

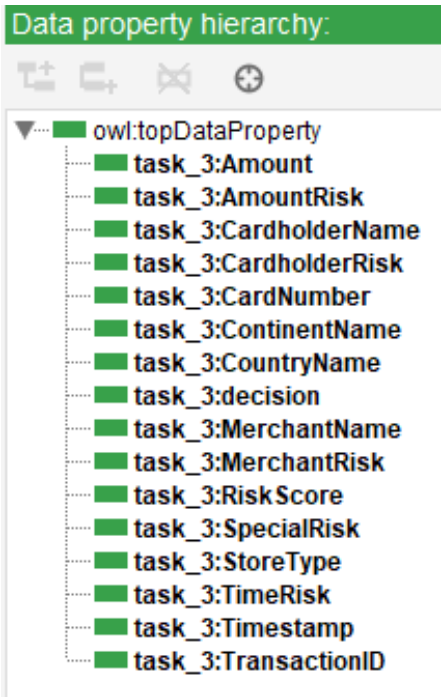

Figure 6.3: The data hierarchy created in the framework protegee

6.2 Define test case

Based on the defined RDF schema, all the necessary individuals were added to the ontology to represent the four scenarios provided in the project description(Figure 6.4), along with some additional cases to verify the correct functioning of the rules. Regarding cardholders, the individuals `mensoni` residing in `italy`, `meyer` residing in `germany`, and `suter` residing in `switzerland` were created. A credit card was associated with each of them, namely `card1`, `card2`, and `card3` respectively, using the `hasCardholder` property. For merchants, `worldOfAfrica` located in `south_africa`, `windsOfTheDesert` located in `morocco`, `mamaAfrica` also located in `south_africa`, and an additional Italian merchant called `mammaMia` were

defined to test queries related to Italian merchants. All merchants were declared as physical stores through the `StoreType` property.

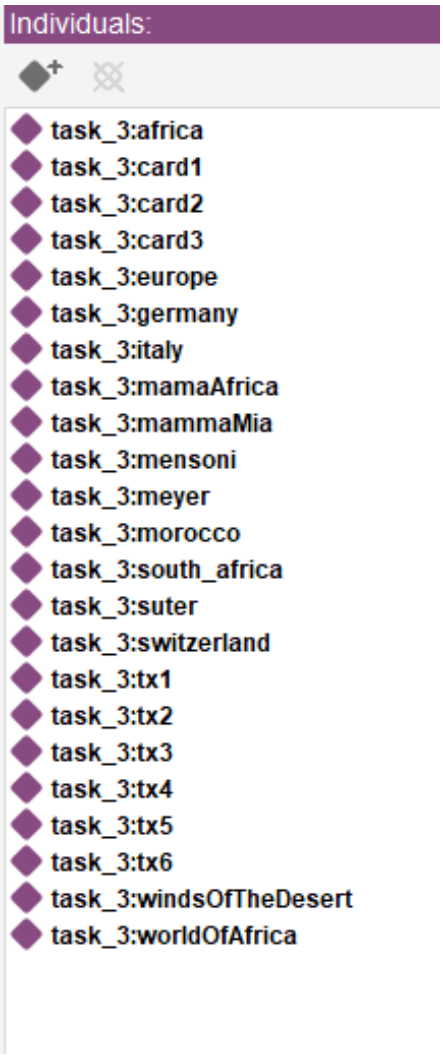

Figure 6.4: All the individuals created in the framework protegee

Six transactions were inserted. Transaction `tx1` represents Mensoni purchasing an item worth 11,234 at the `worldOfAfrica` store in South Africa. Transaction `tx2` represents Meyer paying 12 for a meal at the `windsOfTheDesert` restaurant in Morocco. Transaction `tx3` describes Suter having a 90 at the `mamaAfrica` restaurant in South Africa. To test the five-minute temporal rule, two additional transactions were added using Mensoni's same card: `tx4` occurs 50 seconds after `tx1` at the `windsOfTheDesert` restaurant in Morocco, while `tx5` occurs 1,050 seconds later, thus exceeding the five-minute threshold. Finally, `tx6` represents a 512 transaction by Meyer at the Italian merchant `mammaMia`, useful for verifying queries related to Italian merchants. Each transaction was enriched with `Amount` and `Timestamp` values, the latter expressed in UNIX timestamp format to allow temporal difference calculations. For some transactions, intermediate risk values and the final decision were also precomputed and inserted, as shown by the `AmountRisk`, `CardholderRisk`, `MerchantRisk`, `SpecialRisk`, `TimeRisk`, `RiskScore`, and `decision` properties

present on each individual.
This set of test cases covers all the scenarios expected by the risk rules and allows the system's behaviour to be fully validated.

6.3 SWRL rule

To translate the risk heuristics defined in the previous tasks into formally executable rules, several SWRL rules have been implemented in the ontology. There are three rules for merchant risk. Rule `MerchantRisk1` assigns 60 points if the merchant is located in Africa. Rule `MerchantRisk2` assigns 10 points if the merchant is located in Italy. Rule `MerchantRisk3` assigns 20 points if the merchant is located in Europe but not in Italy.

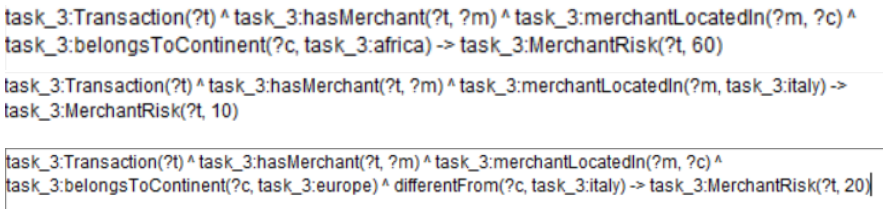

Figure 6.5: The 3 merchant risk rules

Similarly, three rules have been defined for cardholder risk. Rule `CardholderRisk1` assigns 0 points if the cardholder resides in Italy. Rule `CardholderRisk2` assigns 10 points if the cardholder resides in a European country other than Italy. Rule `CardholderRisk3` assigns 20 points if the cardholder resides in Africa.

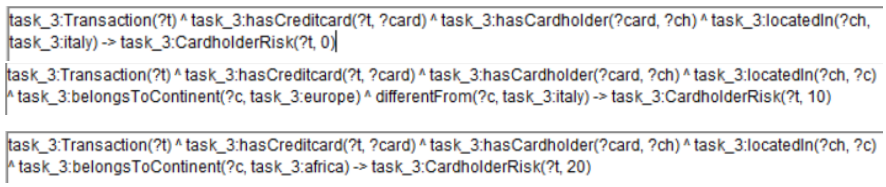

Figure 6.6: The 3 carholder rules

Regarding the transaction amount, the three rules are `AmountRisk1`, which assigns 10 points for amounts below 1000 euros, `AmountRisk2`, which assigns 60 points for amounts between 1000 and 10000 euros, and `AmountRisk3`, which assigns 90 points for amounts above 10000 euros.

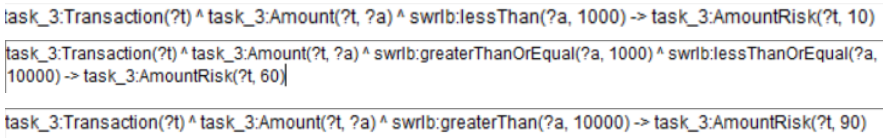

Figure 6.7: The 3 amount risk rules

Special risk is handled by rule `SpecialRisk1`, which assigns 30 points when the cardholder is Swiss and the merchant is located in South Africa, while rule `SpecialRisk2` assigns 0 points in all other cases.

```
task_3:Transaction(?t) ^ task_3:hasCreditcard(?t, ?card) ^ task_3:hasCardholder(?card, ?ch) ^ task_3:locatedIn(?ch, ?switzerland) ^ task_3:hasMerchant(?t, ?m) ^ task_3:merchantLocatedIn(?m, ?country) ^ differentFrom(?country, task_3:south_africa) -> task_3:SpecialRisk(?t, 0)

task_3:Transaction(?t) ^ task_3:hasCreditcard(?t, ?card) ^ task_3:hasCardholder(?card, ?ch) ^ task_3:locatedIn(?ch, task_3:switzerland) ^ task_3:hasMerchant(?t, ?m) ^ task_3:merchantLocatedIn(?m, task_3:south_africa) -> task_3:SpecialRisk(?t, 30)

task_3:Transaction(?t) ^ task_3:hasCreditcard(?t, ?card) ^ task_3:hasCardholder(?card, ?ch) ^ task_3:locatedIn(?ch, ?c) ^ differentFrom(?c, task_3:switzerland) -> task_3:SpecialRisk(?t, 0)
```

Figure 6.8: The 3 special risk rules

The temporal rule, representing the novelty introduced in Task 2, is implemented by `TimeRisk`, which identifies pairs of transactions sharing the same credit card, involving merchants in different countries and of physical type, with a time difference not exceeding 300 seconds, and assigns 150 points in such a case. Rule `TimeRisk2` assigns 0 points when this condition is not met.

```
task_3:Transaction(?t1) ^ task_3:Transaction(?t2) ^ task_3:hasCreditcard(?t1, ?card) ^ task_3:hasCreditcard(?t2, ?card) ^ task_3:hasMerchant(?t1, ?m1) ^ task_3:hasMerchant(?t2, ?m2) ^ task_3:merchantLocatedIn(?m1, ?c1) ^ task_3:merchantLocatedIn(?m2, ?c2) ^ differentFrom(?t1, ?t2) ^ differentFrom(?c1, ?c2) ^ task_3:Timestamp(?t1, ?time1) ^ task_3:Timestamp(?t2, ?time2) ^ swrlb:subtract(?diff, ?time1, ?time2) ^ swrlb:abs(?absdiff, ?diff) ^ swrlb:lessThanOrEqual(?absdiff, 300) -> task_3:TimeRisk(?t1, 150)

task_3:Transaction(?t1) ^ task_3:Transaction(?t2) ^ task_3:hasCreditcard(?t1, ?card) ^ task_3:hasCreditcard(?t2, ?card) ^ task_3:hasMerchant(?t1, ?m1) ^ task_3:hasMerchant(?t2, ?m2) ^ task_3:merchantLocatedIn(?m1, ?c1) ^ task_3:merchantLocatedIn(?m2, ?c2) ^ differentFrom(?t1, ?t2) ^ differentFrom(?c1, ?c2) ^ task_3:Timestamp(?t1, ?time1) ^ task_3:Timestamp(?t2, ?time2) ^ swrlb:subtract(?diff, ?time1, ?time2) ^ swrlb:abs(?absdiff, ?diff) ^ swrlb:lessThanOrEqual(?absdiff, 300) -> task_3:TimeRisk(?t1, 150)
```

Figure 6.9: The 2 time risk rules

Once all partial contributions have been calculated, the `SumRule` sums them using the built-in `add` operator and produces the overall `RiskScore` for each transaction. At this point, three rules determine the final decision: `AcceptedRule` assigns "accepted" if the score is less than or equal to 100, `BlockRule` assigns "stopped" if the score is between 101 and 149, and `RejectRule` assigns "rejected" if the score is greater than or equal to 150.

```
task_3:Transaction(?t) ^ task_3:RiskScore(?t, ?s) ^ swrlb:lessThanOrEqual(?s, 100) -> task_3:decision(?t, "accepted"^^rdf:PlainLiteral)

task_3:Transaction(?t) ^ task_3:RiskScore(?t, ?s) ^ swrlb:greaterThan(?s, 100) ^ swrlb:lessThan(?s, 150) -> task_3:decision(?t, "stopped"^^rdf:PlainLiteral)
```

Figure 6.10: The 2 time risk rules

```
task_3:Transaction(?t) ^ task_3:RiskScore(?t, ?s) ^ swrlb:greaterThanOrEqual(?s, 150) -> task_3:decision(?t, "rejected"^^rdf:PlainLiteral)
```

Figure 6.11: The 3 rule for discrimination each case

6.4 SPARQL query

Before executing the SPARQL queries, it is necessary to declare the following appropriate prefixes to map the namespaces used in the ontology:

```
PREFIX owl: <http://www.w3.org/2002/07/owl#>
```

```
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
PREFIX task_3: <http://www.semanticweb.org/nicola/ontologies/
2026/4/task_3#>
```

The first SPARQL query simply retrieves the complete list of all transactions in the knowledge graph. The query select the variable `?tx` and applies a pattern that requires that `?tx` is of type `Transaction`. This query is useful for getting an overview of the data loaded into the ontology and for verifying that all individuals were imported correctly.

```
#1 all transactions

SELECT ?t
WHERE {
  ?t rdf:type task_3:Transaction .
}
```

?transaction
task_3:tx6
task_3:tx5
task_3:tx4
task_3:tx3
task_3:tx2
task_3:tx1

Figure 6.12: The result of the first query

The second query extracts transactions whose merchant is based in South Africa. To achieve this, the query uses two patterns: the first binds the transaction `?tx` to his merchant `?m` through the `hasMerchant` property, while the second requires that the merchant `?m` is located in South Africa via the property `merchantLocatedIn` , which points to the individual `south_africa`. This query makes it possible to quickly identify all transactions involving South African traders, which, according to the risk rules, result in an increase of 60 points.

```
#2 transactions with merchants from South

Africa
SELECT ?t ?m
WHERE {
  ?t rdf:type task_3:Transaction ;
     task_3:hasMerchant ?m .
  ?m task_3:merchantLocatedIn task_3:south_africa .
}
```

The third query retrieves transactions whose cardholders reside in a European country. Graph navigation occurs through a chain of properties: from the transaction `?tx` do we go back to the credit card `?card` via `hasCreditcard`, from the card you get

?transaction	?merchant
task_3:tx3	task_3:mamaAfrica
task_3:tx1	task_3:worldOfAfrica

Figure 6.13: The result of the second query

the cardholder `?ch` with `hasCardholder`, from the cardholder you access the country of residence `?c` with `locatedIn`, and finally it is verified that this country belongs to the European continent by `belongsToContinent` which points to the individual `europe`. This query is particularly useful for analyzing the behavior of European cardholders, who contribute to risk differently depending on whether they reside in Italy or other European countries.

```
#3 Transactions with cardholders from Europe

SELECT ?t ?ch
WHERE {
  ?t rdf:type task_3:Transaction ;
     task_3:hasCreditcard ?card .
  ?card task_3:hasCardholder ?ch .
  ?ch task_3:locatedIn ?c .
  ?c task_3:belongsToContinent task_3:europe .
}
```

?transaction	?cardHolder
task_3:tx6	task_3:meyer
task_3:tx5	task_3:suter
task_3:tx4	task_3:mensoni
task_3:tx3	task_3:suter
task_3:tx2	task_3:meyer
task_3:tx1	task_3:mensoni

Figure 6.14: The result of the third query

The fourth query extracts transactions whose merchants are based in Italy, showing for each of them the value of the risk associated with the merchant. The query bind the transaction `?tx` to his merchant `?m` via `hasMerchant`, verifies that the merchant is located in Italy with `merchantLocatedIn` pointing to `italy`, and retrieves the merchant's risk value through the `MerchantRisk` data property associated with the transaction.

```
#4 Transactions and their merchant risk for transactions with
merchant from Italy

SELECT ?t ?mr
WHERE {
  ?t rdf:type task_3:Transaction ;
     task_3:hasMerchant ?m ;
     task_3:MerchantRisk ?mr .
  ?m task_3:merchantLocatedIn task_3:italy .
}
```

?transaction	?totalRiskScore
task_3:tx5	230
task_3:tx4	220
task_3:tx3	260
task_3:tx1	300

Figure 6.15: The result of the fourth query

6.5 Special SPARQL query

After activating the reasoner that applies all SWRL rules, the final query retrieves transactions considered fraudulent based on the risk score. The query selects the transaction `?t` and its `RiskScore` ?score, requiring that the transaction is of type `Transaction` and that the risk score is greater than 100. This approach is more direct than filtering on the decision, since the threshold of 100 points represents the limit beyond which a transaction is no longer automatically accepted. Thus, both stopped transactions (with a score between 101 and 150) and rejected transactions (with a score above 150) are selected.

```
#5 Retrieve the fraudulent transactions.

SELECT ?t ?score
WHERE {
  ?t rdf:type task_3:Transaction ;
     task_3:RiskScore ?score .
  FILTER(?score > 100)
}
```

?transactions	?merchant_risk
task_3:tx6	10

Figure 6.16: The result of the fifth query

7. A-B Transaction

7.1 Additional use cases

This section introduces two new transaction scenarios required to validate the classification rules for A-transactions and B-transactions defined in Part 5 of the project specification. An A-transaction is any transaction made in an online store, while a B-transaction is defined as the complement: any transaction that is not an A-transaction, i.e., any transaction performed in a physical store.

Scenario 5: Mr. Miller

Mr. Miller is a German cardholder ordering an item worth €102 from the online store "South African Master Pieces", located in South Africa.

Sub-decision	**Score**
`Merchant_Risk` (South Africa → Africa)	60
`Carholder_Risk` (Germany → Europe)	10
`Special_Risk` (Germany + South Africa)	0
`Amount_Risk` (€102 < €1,000)	10
`Time_Risk`	0
Total_Risk	**80**

Table 7.1: Risk score breakdown for Mr. Miller.

The total score of 80 does not exceed 100, resulting in an `Accepted` outcome. Since the transaction is performed in an online store, it is classified as an **A-transaction**.

Scenario 6: Mr. Suter (revisited)

Mr. Suter is a Swiss cardholder paying for a meal at the restaurant "Mama Africa" in South Africa for €90. This scenario was already introduced in Task 1 as Scenario 3; it is reused here to verify the B-transaction classification, since the restaurant operates as a physical store.

The total score of 100 satisfies the condition ≤ 100, resulting in an `Accepted` outcome. Since the transaction is performed in a physical store and is therefore not an A-transaction, it is classified as a **B-transaction**.

Sub-decision	Score
`Merchant_Risk` (South Africa → Africa)	60
`Carholder_Risk` (Switzerland → Other)	0
`Special_Risk` (Switzerland + South Africa)	30
`Amount_Risk` (€90 < €1,000)	10
`Time_Risk`	0
Total_Risk	**100**

Table 7.2: Risk score breakdown for Mr. Suter (B-transaction scenario).

7.2 Prolog Updates

The predicates

For the Prolog part, only a few updates were needed to meet the assignment requirements, since the environment was already set up. Notably, a `StoreType` attribute was already present in each `merchant` fact from the previous implementation, which simplified the task considerably. The only addition required was the definition of two clauses for the predicate `transactionType/2`.

Recalling the definitions from the project specification, an A-transaction is defined as follows: *"Every transaction made in an online store is classified as an A-transaction"*, while a B-transaction is defined as its complement: *"A B-transaction is the opposite of an A-transaction"*, i.e., every transaction that is not an A-transaction.

Retrieve A-transaction To identify an A-transaction, it is sufficient to verify that the merchant associated with the transaction operates as an online store:

```
transactionType(Trans, a) :-
    transaction(Trans, _, MerchantID, _, _),
    merchant(MerchantID, _, _, online).
```

Retrieve B-transaction Although it may seem that checking for a physical store type would be sufficient, the specification defines B-transactions strictly as the negation of A-transactions. It is therefore necessary to use the `\+` operator, which implements *negation as failure*: it succeeds if the given goal cannot be proved, which fits precisely with the intended semantics.

```
transactionType(Trans, b) :-
    transaction(Trans, _, _, _, _),
    \+ transactionType(Trans, a).
```

A transaction is therefore classified as a B-transaction if and only if it exists in the knowledge base and cannot be proved to be an A-transaction.

Query results

The `transactionType/2` predicate supports two usage patterns: checking the type of a specific transaction, or retrieving all transactions of a given type.

Check if A or B To determine the type of a specific transaction, the predicate is queried with the transaction identifier as a constant and a free variable in the second argument:

```
transactionType(tx1, X).
```

The variable `X` will be unified with either `a` or `b`, returning the type of the selected transaction.

List all A-transactions or B-transactions To retrieve all transactions of a given type, the transaction identifier is left as a free variable:

```
transactionType(X, a).
transactionType(X, b).
```

In the current knowledge base, `tx6` is the only A-transaction, as it is the only one associated with an online merchant. The remaining five transactions (`tx1` through `tx5`) are all classified as B-transactions, since they are performed at physical stores.

7.3 SWRL rules

Additions from the previous version

Before discussing the SWRL rules, it is useful to describe the new elements added to the Protégé file to address this task. First, A-transaction and B-transaction were added as subclasses of Transaction, as required by the assignment specification.

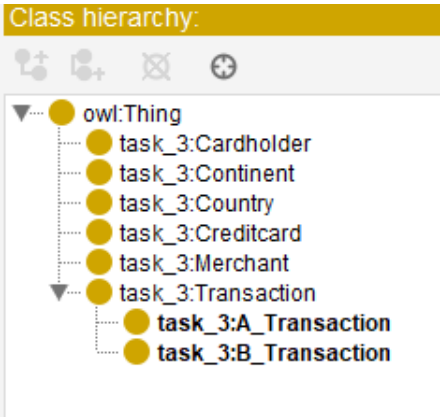

Figure 7.1: The updated Protégé class hierarchy with the two new subclasses.

Figure 7.2 shows the ontology graph, which visually represents the relationships between the classes defined in the schema, including the newly introduced A-transaction and B-transaction.

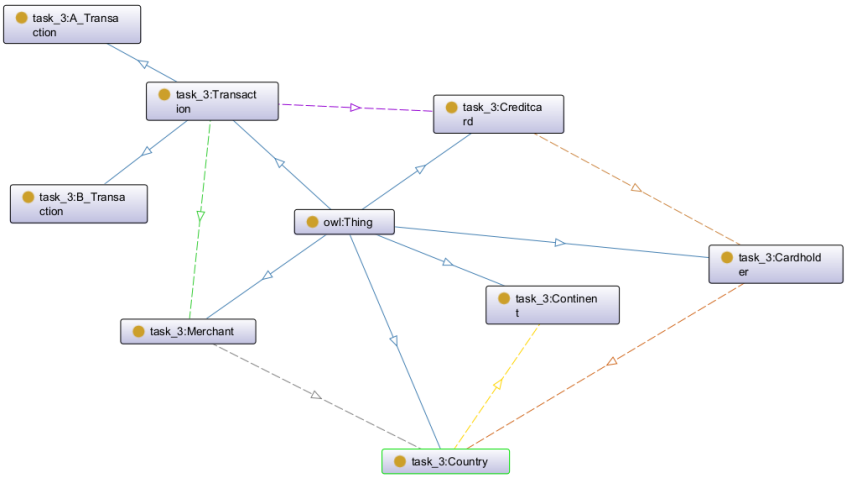

Figure 7.2: Ontology graph

A new online merchant, "South African Master Pieces", was added to the individuals list. Two new transactions were also introduced to cover the scenarios defined in Part 5 of the assignment.

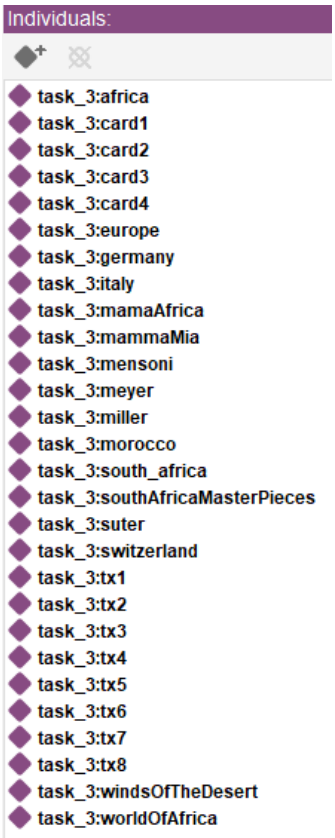

Figure 7.3: The updated Protégé individuals list with the newly added elements.

The SWRL rules

Before presenting the proposed solution, the question posed by the assignment must be addressed directly:

Is it possible to create SWRL rules to identify "A-transaction" and "B-transaction"?

The answer is **no**, at least not in a strict sense. The reason lies in the nature of SWRL and OWL, which operate under the Open World Assumption (OWA) and do not support negation as failure. As a consequence, while it is straightforward to derive all A-transactions by checking whether the associated merchant operates as an online store, it is not possible to formally derive all B-transactions as the complement of A-transactions: the absence of a proof that a transaction is an A-transaction cannot be interpreted as evidence that it is a B-transaction.

However, if one accepts the pragmatic assumption that every transaction in the knowledge base is associated with a merchant whose store type is explicitly declared (either `online` or `physical`) then a rule can be defined that directly checks for the `physical` store type, effectively recovering the intended semantics without relying on negation.

This is the approach adopted in the implementation. Rather than deriving B-transactions as "not A", the two rules explicitly match on the `StoreType` data property, assigning each transaction to the appropriate subclass.

```
task_3:Transaction(?t) ^ task_3:hasMerchant(?t, ?m) ^ task_3:StoreType(?m, "online"^^rdf:PlainLiteral) ->
task_3:A_Transaction(?t)
task_3:Transaction(?t) ^ task_3:hasMerchant(?t, ?m) ^ task_3:StoreType(?m, "physical"^^rdf:PlainLiteral) ->
task_3:B_Transaction(?t)
```

Figure 7.4: The SWRL rules for assigning the A-transaction and B-transaction subclasses to each transaction individual.

With this compromise, all transactions in the knowledge base can be correctly assigned to their respective subclass. This solution was designed primarily to support the SPARQL retrieval query discussed in the following section.

7.4 SPARQL retrieve query

As in the previous section, some clarification must be provided before presenting the queries. Similarly to SWRL, SPARQL does not natively support negation as failure, which makes the retrieval of B-transactions apparently unsuitable. However, unlike the SWRL case, SPARQL does offer a workaround: through the combined use of `OPTIONAL` and `FILTER(BOUND(...))`, it is possible to simulate negation for this specific case, as described below.

Retrieve A-transactions The retrieval of A-transactions is straightforward: the query simply selects all individuals that belong to the `A_Transaction` subclass, which was populated by the SWRL rule discussed in the previous section.

```
# 6. Retrieve all transactions belonging to A_Transaction
SELECT ?transaction
WHERE {
```

```
    ?transaction rdf:type task_3:A_Transaction .
}
```

Based on the current knowledge base, the only result is `tx7`, which is the only transaction associated with an online merchant.

?transaction
task_3:tx7

Figure 7.5: The result of the A_Transaction retrieval query.

Retrieve B-transactions — first solution For B-transactions, the simplest approach mirrors the compromise adopted in the SWRL section: since the `B_Transaction` subclass has been explicitly populated by the corresponding SWRL rule, it can be queried directly.

```
# 7.a Retrieve all transactions belonging to B_Transaction
SELECT ?transaction
WHERE {
            ?transaction rdf:type task_3:B_Transaction .
}
```

As noted previously, this solution is only a compromise to face that problem but it is not strictly close to the B_type description.

Retrieve B-transactions — second solution A more principled solution can be achieved using the `OPTIONAL` and `FILTER` clauses to simulate negation within SPARQL. The query proceeds in three logical steps:

```
# 7.b Retrieve all transactions belonging to B_Transaction
# using OPTIONAL/FILTER negation pattern

SELECT ?transaction
WHERE {
    # Step 1: select all transactions
    ?transaction a task_3:Transaction .

    # Step 2: optionally bind typeA if the transaction is an A_Transaction
    OPTIONAL {
        ?transaction rdf:type
         task_3:A_Transaction.
    }

    # Step 3: retain only transactions where typeA was not bound
    FILTER (!BOUND(?typeA))
}
```

In Step 1, all individuals of type `Transaction` are selected. In Step 2, the `OPTIONAL` block attempts to match each transaction against the `A_Transaction` subclass; if the

match succeeds, the variable `?typeA` is bound. In Step 3, the `FILTER` clause retains only those transactions for which `?typeA` was never bound, effectively selecting all transactions that are not A-transactions.

Both solutions produce the same result: all transactions except `tx7` are returned, since the remaining ones are all associated with physical stores and are therefore classified as B-transactions. The main difference is that the second solution does not require the `B_Transaction` subclass to be explicitly asserted in the ontology: it retrieves B-transactions indirectly, by selecting all transactions that are *not* classified as `A_Transaction`. The first solution, by contrast, relies on `B_Transaction` being defined and populated. The second approach is therefore more robust and generally preferable.

?transaction
task_3:tx8
task_3:tx6
task_3:tx5
task_3:tx4
task_3:tx3
task_3:tx2
task_3:tx1

Figure 7.6: The result of both the B_Transaction retrieval queries.

8. Conclusion

This Knowledge Engineering project concerns a prototype of a fraud detection application developed across three different paradigms: DMN, Prolog, and OWL/Protégé.

8.1 Conclusion About DMN

The DMN model proved to be an effective tool for representing the static, per-transaction risk-scoring logic required by the project. The decomposition into independent sub-decisions (Amount Risk, Merchant Risk, Cardholder Risk, Special Risk), supported by the Country_Continent BKM, made the model modular, readable, and easy to validate against the predefined test scenarios.
However, the implementation of Rule 5 highlighted a structural limitation of the formalism: DMN evaluates a single decision instance in isolation and has no native mechanism for accessing historical data or comparing transactions over time. Representing the 5-minute rule therefore required moving the temporal check outside the DMN engine, delegating it to an external database query and injecting its result as a boolean input (TimeFlag). This shows that, while DMN is well suited for stateless, rule-based decisions, it must be embedded in a broader architecture whenever temporal or relational reasoning across multiple records is required.

8.2 Prolog conclusion

The Prolog implementation made it possible to express the entire fraud detection logic, including the temporal rule, within a single declarative knowledge base. The risk-scoring predicates (`merchant_risk`, `cardholder_risk`, `amount_risk`, `special_risk`) mirror the DMN sub-decisions almost one-to-one, while `score_risk_comparison` and `time_comparison` integrate the temporal check directly, without the need for an external component as in DMN.
The logic underlying Prolog also simplified the A/B transaction classification: since `transactionType(Trans, b)` could be defined simply as the negation as failure of `transactionType(Trans, a)`, the complementarity required by the specification was captured in a single, intuitive rule. This contrasts sharply with the difficulties encountered in the OWL/SWRL representation of the same problem.

8.3 Knowledge Graph and OWL conclusion

The translation of the domain into an OWL ontology, populated with RDF individuals, provided a formal and reusable representation of the fraud detection domain (Trans-

action, Cardholder, Merchant, Credit card, Country, Continent, and their relations). The SWRL rules successfully replicated all the scoring logic defined in the previous tasks, and after activating the reasoner, SPARQL queries could retrieve transactions of interest, including the fraudulent ones, simply by filtering on the inferred RiskScore. The A/B transaction task, however, exposed the main limitation of this formalism: under the Open World Assumption, neither SWRL nor SPARQL can natively derive a class as the complement of another. Deriving B_Transaction as "not an A_Transaction" is not possible in a strict sense, since the absence of a proof is not equivalent to a proof of absence. The adopted workaround — explicitly asserting B_Transaction based on the StoreType property, and retrieving B-transactions in SPARQL via the `OPTIONAL/FILTER(!BOUND(...))` pattern — made it possible to obtain the expected results, but only by relying on a pragmatic, closed-world-like assumption about the completeness of the data.

8.4 General conclusion

Overall, the project illustrates how the same fraud detection logic can be progressively reformulated across formalisms with very different underlying assumptions. DMN offers the clearest, most business-oriented representation but is inherently stateless; Prolog, thanks to the Closed World Assumption and negation as failure, handles both scoring and temporal/complementary reasoning naturally within a single knowledge base; OWL/SWRL/SPARQL provide a standardized, interoperable, and inference-capable representation, at the cost of additional care whenever negative or complementary concepts (such as B-transactions) need to be expressed under the Open World Assumption.